# Informe — Práctica de Laboratorio Neo4j

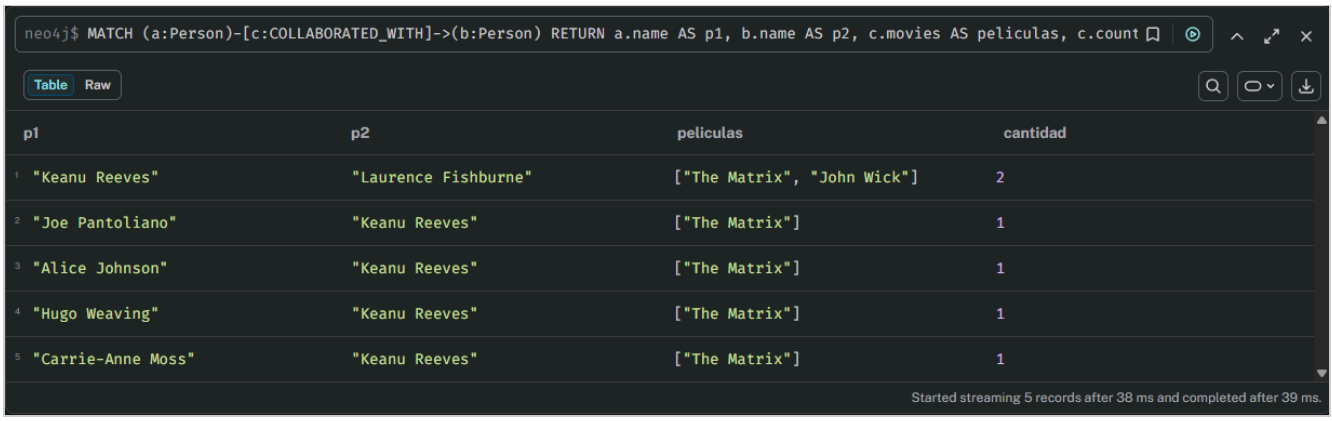
## Bases de Datos II — Prof. Kenneth Obando Rodríguez

## 1. Ambiente

Neo4j Desktop 2.2.1, instancia local `practica-neo4j`, base `neo4j`, usuario `neo4j`, Cypher 25. Datos generados con IA según autorización del docente.

## 2. Carga de datos

Constraints de unicidad en `Movie.title` y `Person.name`. 36 nodos (24 Person, 12 Movie), 48 relaciones (ACTED_IN, DIRECTED, PRODUCED, WROTE, REVIEWED, FOLLOWS).




## 3. Resultados Q1–Q18 (resumen)

Consultas completas en `practica_neo4j.cypher`. Tablas en `Qn.csv`, capturas en `evidencias/Qn.png`.

### Q1 Películas por década — 7 filas

```
neo4j$ MATCH (m:Movie) WHERE m.released >= 1990 AND m.released <= 1999 RETURN m.title AS titulo, m.released AS anno, m.tagl
```

Table | Raw

| | titulo | anno | tag |
|---|---|---|---|
| 1 | "A Few Good Men" | 1992 | "In the heart of the nation" |
| 2 | "Forrest Gump" | 1994 | "Life is like a box of chocolates" |
| 3 | "Pulp Fiction" | 1994 | "Character driven story" |
| 4 | "Speed" | 1994 | "Get ready for rush hour" |
| 5 | "Apollo 13" | 1995 | "Houston we have a problem" |
| 6 | "Toy Story" | 1995 | "To infinity and beyond" |
| 7 | "The Matrix" | 1999 | "Welcome to the Real World" |

Started streaming 7 records after 36 ms and completed after 36 ms.

## Q2 Reparto The Matrix — 6 actores

```
neo4j$ MATCH (p:Person)-[r:ACTED_IN]->(m:Movie) WHERE m.title = "The Matrix" RETURN p.name, r.roles
```

Table | Raw

| | p.name | r.roles |
|---|---|---|
| 1 | "Keanu Reeves" | ["Neo"] |
| 2 | "Laurence Fishburne" | ["Morpheus"] |
| 3 | "Carrie-Anne Moss" | ["Trinity"] |
| 4 | "Hugo Weaving" | ["Agent Smith"] |
| 5 | "Joe Pantoliano" | ["Cypher"] |
| 6 | "Alice Johnson" | ["Background Agent"] |

Started streaming 6 records after 54 ms and completed after 54 ms.

## Q3 Películas con directores — 12

```
neo4j$ MATCH (m:Movie) OPTIONAL MATCH (p:Person)-[:DIRECTED]->(m) RETURN m.title, collect(p.name) AS directores
```

Table | Raw

| | m.title | directores |
|---|---|---|
| 1 | "The Matrix" | ["Lana Wachowski", "Lilly Wachowski"] |
| 2 | "John Wick" | [] |
| 3 | "Speed" | [] |
| 4 | "Forrest Gump" | ["Robert Zemeckis"] |
| 5 | "Apollo 13" | ["Ron Howard"] |
| 6 | "Cast Away" | [] |
| 7 | "A Few Good Men" | [] |
| 8 | "Footloose" | [] |
| 9 | "Cloud Atlas" | ["Lana Wachowski"] |
| 10 | "Man of Tai Chi" | ["Keanu Reeves"] |

Started streaming 12 records after 57 ms and completed after 59 ms.

## Q4 Nacidos después de 1965 — 3

```
neo4j$ MATCH (p:Person)-[:ACTED_IN]->(m:Movie) WHERE p.born > 1965 WITH p, count(m) AS cantidadPeliculas WHERE cantidadPeli
```

Table | Raw

| | p.name | p.born | cantidadPeliculas |
|---|---|---|---|
| 1 | "Carrie-Anne Moss" | 1967 | 1 |
| 2 | "Robin Wright" | 1966 | 1 |
| 3 | "Alice Johnson" | 1990 | 1 |

Started streaming 3 records after 72 ms and completed after 77 ms.

## Q5 Colaboradores Keanu — 6

```
neo4j$ MATCH (keanu:Person)-[:ACTED_IN]->(m:Movie)<-[:ACTED_IN]-(colega:Person) WHERE keanu.name = "Keanu Reeves" RETURN cc
```

Table | Raw

| | colega.name | collect(m.title) |
|---|---|---|
| 1 | "Laurence Fishburne" | ["The Matrix", "John Wick"] |
| 2 | "Carrie-Anne Moss" | ["The Matrix"] |
| 3 | "Hugo Weaving" | ["The Matrix"] |
| 4 | "Joe Pantoliano" | ["The Matrix"] |
| 5 | "Alice Johnson" | ["The Matrix"] |
| 6 | "Sandra Bullock" | ["Speed"] |

Started streaming 6 records after 62 ms and completed after 62 ms.

## Q6 Múltiples funciones — 5

```
neo4j$ MATCH (p:Person)-[r:ACTED_IN|DIRECTED|PRODUCED|WROTE]->(m:Movie) WITH p, collect(DISTINCT type(r)) AS conexiones, cc
```

Table | Raw

| | p.name | conexiones | peliculas | cantidadPeliculas |
|---|---|---|---|---|
| 1 | "Keanu Reeves" | ["ACTED_IN", "PRODUCED", "DIRECTED"] | ["The Matrix", "John Wick", "Speed", "Man of Tai Chi"] | 4 |
| 2 | "Lana Wachowski" | ["DIRECTED", "WROTE"] | ["The Matrix", "Cloud Atlas"] | 2 |
| 3 | "Lilly Wachowski" | ["DIRECTED", "WROTE"] | ["The Matrix", "Cloud Atlas"] | 2 |
| 4 | "Tom Hanks" | ["ACTED_IN", "PRODUCED", "WROTE"] | ["Forrest Gump", "Apollo 13", "Cast Away", "Cloud Atlas", "Toy Story"] | 5 |
| 5 | "Gary Sinise" | ["ACTED_IN", "PRODUCED"] | ["Forrest Gump", "Apollo 13"] | 2 |

Started streaming 5 records after 65 ms and completed after 68 ms.

## Q7 Reparto amplio — The Matrix (6)

```
neo4j$ MATCH (p:Person)-[:ACTED_IN]->(m:Movie) WITH m, collect(p.name) AS actores WHERE size(actores) >= 5 RETURN m.title,
```

Table | Raw

| m.title | m.released | cantidadActores | actores |
|---|---|---|---|
| 1 "The Matrix" | 1999 | 6 | ["Keanu Reeves", "Laurence Fishburne", "Carrie-Anne Moss", "Hugo Weaving", "Joe Pantoliano", "Alice Johnson"] |

Started streaming 1 record after 53 ms and completed after 56 ms.

## Q8 Seguidos y reseñas — 4

```
neo4j$ MATCH (seguidor:Person)-[:FOLLOWS]->(seguido:Person)-[r:REVIEWED]->(m:Movie) OPTIONAL MATCH (seguidor)-[r2:REVIEWED]
```

Table | Raw

| quienSigue | aQuienSigue | pelicula | calificacion | seguidorTambienReseno |
|---|---|---|---|---|
| 1 "James Thompson" | "Alice Johnson" | "The Matrix" | 5 | TRUE |
| 2 "James Thompson" | "Alice Johnson" | "Forrest Gump" | 4 | FALSE |
| 3 "James Thompson" | "Bob Smith" | "Forrest Gump" | 5 | FALSE |
| 4 "James Thompson" | "Bob Smith" | "Apollo 13" | 4 | FALSE |

Started streaming 4 records after 80 ms and completed after 85 ms.

## Q9 Ruta Keanu–Hanks — 4 saltos

```
neo4j$ MATCH (a:Person {name: "Keanu Reeves"}), (b:Person {name: "Tom Hanks"}) MATCH ruta = shortestPath((a)-[:ACTED_IN|DIF
```

Table | Raw

| nodosDeLaRuta | saltos |
|---|---|
| 1 ["Keanu Reeves", "The Matrix", "Hugo Weaving", "Cloud Atlas", "Tom Hanks"] | 4 |

Started streaming 1 record after 94 ms and completed after 102 ms.

**2 info messages**

**03N90: Cartesian product**

The disconnected pattern '(a:Person {name: 'Keanu Reeves'}), (b:Person {name: 'Tom Hanks'})' builds a cartesian product. A cartesian product may produce a large amount of data and slow down query processing.

```
MATCH (a:Person {name: "Keanu Reeves"}), (b:Person {name: "Tom Hanks"})
^
MATCH ruta = shortestPath((a)-[:ACTED_IN|DIRECTED|PRODUCED|WROTE*]-(b))
RETURN [n IN nodes(ruta) | coalesce(n.name, n.title)] AS nodosDeLaRuta,
       length(ruta) AS saltos
```

**03N91: Unbounded variable length pattern**

The provided pattern '(a)-[:ACTED_IN|DIRECTED|PRODUCED|WROTE*]-(b)' is unbounded. Shortest path with an unbounded pattern may result in long execution times. Use an upper limit (e.g. '[*..5]') on the number of node hops in your pattern.

```
MATCH (a:Person {name: "Keanu Reeves"}), (b:Person {name: "Tom Hanks"})
MATCH ruta = shortestPath((a)-[:ACTED_IN|DIRECTED|PRODUCED|WROTE*]-(b))
                          ^
RETURN [n IN nodes(ruta) | coalesce(n.name, n.title)] AS nodosDeLaRuta,
```

## Q10 Recomendación James — Forrest Gump 4.5, Apollo 13 4.0

```
neo4j$ MATCH (james:Person {name: "James Thompson"})-[:FOLLOWS]->(seguido:Person)-[r:REVIEWED]->(m:Movie) WHERE NOT (james)
```

Table | Raw

| | peliculaRecomendada | recomendadaPor | calificacionPromedio |
|---|---|---|---|
| 1 | "Forrest Gump" | ["Alice Johnson", "Bob Smith"] | 4.5 |
| 2 | "Apollo 13" | ["Bob Smith"] | 4.0 |

Started streaming 2 records after 127 ms and completed after 136 ms.

## Q11 Red colaboración Top10

```
neo4j$ MATCH (actor:Person)-[:ACTED_IN]->(m:Movie)<-[:ACTED_IN]-(colega:Person) WHERE actor <> colega RETURN actor.name AS
```

Table | Raw

| | actor | colaboradoresDistintos |
|---|---|---|
| 1 | "Keanu Reeves" | 6 |
| 2 | "Hugo Weaving" | 6 |
| 3 | "Tom Hanks" | 6 |
| 4 | "Laurence Fishburne" | 5 |
| 5 | "Alice Johnson" | 5 |
| 6 | "Joe Pantoliano" | 5 |
| 7 | "Carrie-Anne Moss" | 5 |
| 8 | "Sally Field" | 3 |
| 9 | "Robin Wright" | 3 |
| 10 | "Gary Sinise" | 3 |

Started streaming 10 records after 65 ms and completed after 102 ms.

## Q12 Bacon — 12 alcanzables

| persona | numeroDeBacon |
|---|---|
| "Gary Sinise" | 2 |
| "Hugo Weaving" | 2 |
| "Robin Wright" | 2 |
| "Sally Field" | 2 |
| "Alice Johnson" | 3 |
| "Carrie-Anne Moss" | 3 |
| "Joe Pantoliano" | 3 |
| "Keanu Reeves" | 3 |
| "Laurence Fishburne" | 3 |
| "Sandra Bullock" | 4 |

Started streaming 12 records after 105 ms and completed after 122 ms.

**2 info messages**

**03N90: Cartesian product**

The disconnected pattern '(bacon:Person {name: 'Kevin Bacon'}), (p:Person)' builds a cartesian product. A cartesian product may produce a large amount of data and slow down query processing.

```
MATCH (bacon:Person {name: "Kevin Bacon"}), (p:Person)
^
WHERE p <> bacon
MATCH ruta = shortestPath((bacon)-[:ACTED_IN*]-(p))
RETURN p.name AS persona,
       length(ruta) / 2 AS numeroDeBacon
ORDER BY numeroDeBacon ASC, persona ASC
LIMIT 20
```

**03N91: Unbounded variable length pattern**

The provided pattern '(bacon)-[:ACTED_IN*]-(p)' is unbounded. Shortest path with an unbounded pattern may result in long execution times. Use an upper limit (e.g. '[*..5]') on the number of node hops in your pattern.

```
MATCH (bacon:Person {name: "Kevin Bacon"}), (p:Person)
```

## Q13 Actuaron y dirigieron — Keanu

```
neo4j$ MATCH (p:Person)-[:ACTED_IN]->(peliActuada:Movie) WITH p, collect(DISTINCT peliActuada.title) AS peliculasActuadas
```

Table Raw

| persona | peliculasActuadas | peliculasDirigidas |
|---|---|---|
| "Keanu Reeves" | ["The Matrix", "John Wick", "Speed"] | ["Man of Tai Chi"] |

Started streaming 1 record after 55 ms and completed after 57 ms.

## Q14 Múltiples roles — 2

```
neo4j$ MATCH (p:Person)-[r:ACTED_IN]->(m:Movie) WHERE size(r.roles) > 1 RETURN p.name AS actor, m.title AS pelicula, r.role
```

Table Raw

| actor | pelicula | roles | cantidadRoles |
|---|---|---|---|
| "Hugo Weaving" | "Cloud Atlas" | ["Bill Smoke", "Haskell Moore"] | 2 |
| "Tom Hanks" | "Cloud Atlas" | ["Dr Goose", "Hotel Manager", "Isaac Sachs"] | 3 |

Started streaming 2 records after 45 ms and completed after 46 ms.

## Q15 Similares — John Wick/The Matrix (2)

```
neo4j$ MATCH (m1:Movie)<-[:ACTED_IN]-(p:Person)-[:ACTED_IN]->(m2:Movie) WHERE m1.title < m2.title WITH m1, m2, collect(DIST
```

Table | Raw

| | película1 | película2 | actoresCompartidos | similitud |
|---|---|---|---|---|
| 1 | "John Wick" | "The Matrix" | ["Keanu Reeves", "Laurence Fishburne"] | 2 |

Started streaming 1 record after 87 ms and completed after 90 ms.

### Q16 Materialización — 30 relaciones

```
neo4j$ MATCH (a:Person)-[:ACTED_IN]->(m:Movie)<-[:ACTED_IN]-(b:Person) WHERE a.name < b.name WITH a, b, collect(DISTINCT m.
```

Created 30 relationships, set 60 properties

Completed after 110 ms

### Q17 Top colaboraciones

```
neo4j$ MATCH (a:Person)-[c:COLLABORATED_WITH]->(b:Person) RETURN a.name AS persona1, b.name AS persona2, c.count AS peliculas
```

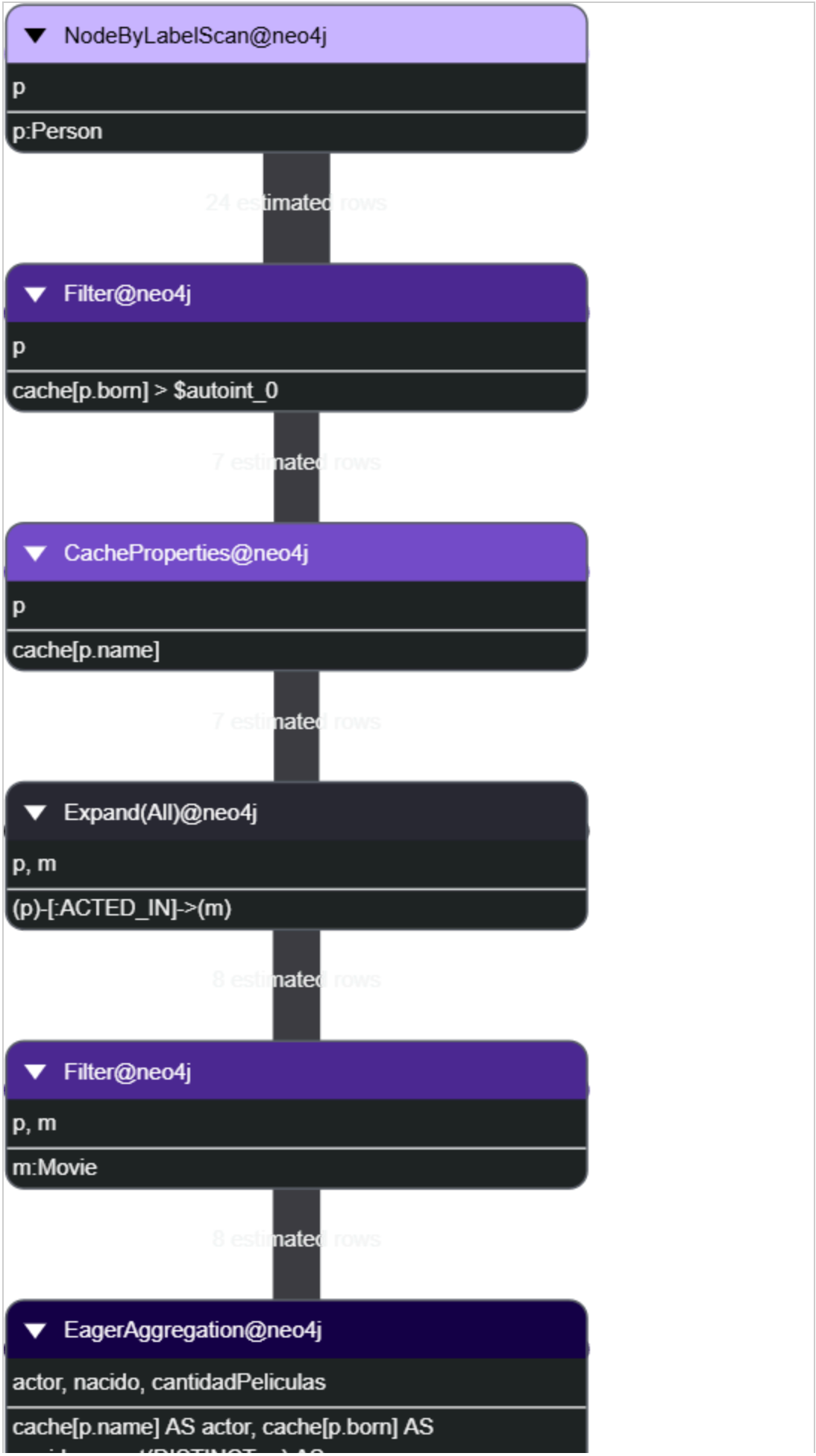
Table | Raw

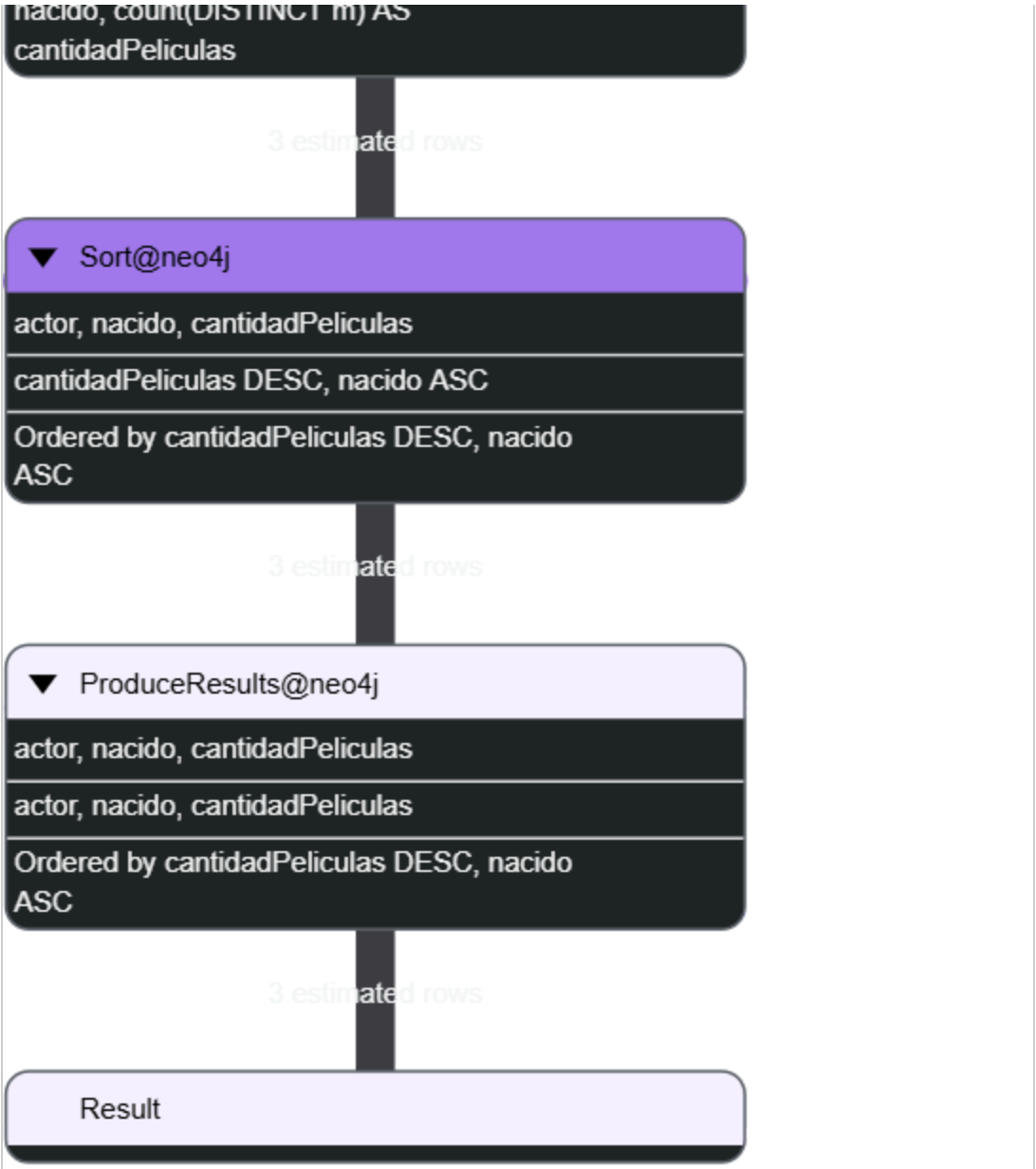
| | persona1 | persona2 | peliculasCompartidas | peliculas |
|---|---|---|---|---|
| 1 | "Keanu Reeves" | "Laurence Fishburne" | 2 | ["The Matrix", "John Wick"] |
| 2 | "Joe Pantoliano" | "Keanu Reeves" | 1 | ["The Matrix"] |
| 3 | "Joe Pantoliano" | "Laurence Fishburne" | 1 | ["The Matrix"] |
| 4 | "Alice Johnson" | "Carrie-Anne Moss" | 1 | ["The Matrix"] |
| 5 | "Carrie-Anne Moss" | "Laurence Fishburne" | 1 | ["The Matrix"] |
| 6 | "Hugo Weaving" | "Laurence Fishburne" | 1 | ["The Matrix"] |
| 7 | "Alice Johnson" | "Keanu Reeves" | 1 | ["The Matrix"] |
| 8 | "Alice Johnson" | "Laurence Fishburne" | 1 | ["The Matrix"] |
| 9 | "Hugo Weaving" | "Keanu Reeves" | 1 | ["The Matrix"] |
| 10 | "Carrie-Anne Moss" | "Keanu Reeves" | 1 | ["The Matrix"] |

Started streaming 10 records after 35 ms and completed after 37 ms.

## 4. Q18 EXPLAIN / PROFILE

Q4 (media): filtro + agregación simple, bajo costo. Q11 (difícil): doble Expand + count DISTINCT + sort, costo mayor. Mejoras: usar índices de constraints, evitar cartesiano, acotar `*..10`, filtrar `<>` temprano.

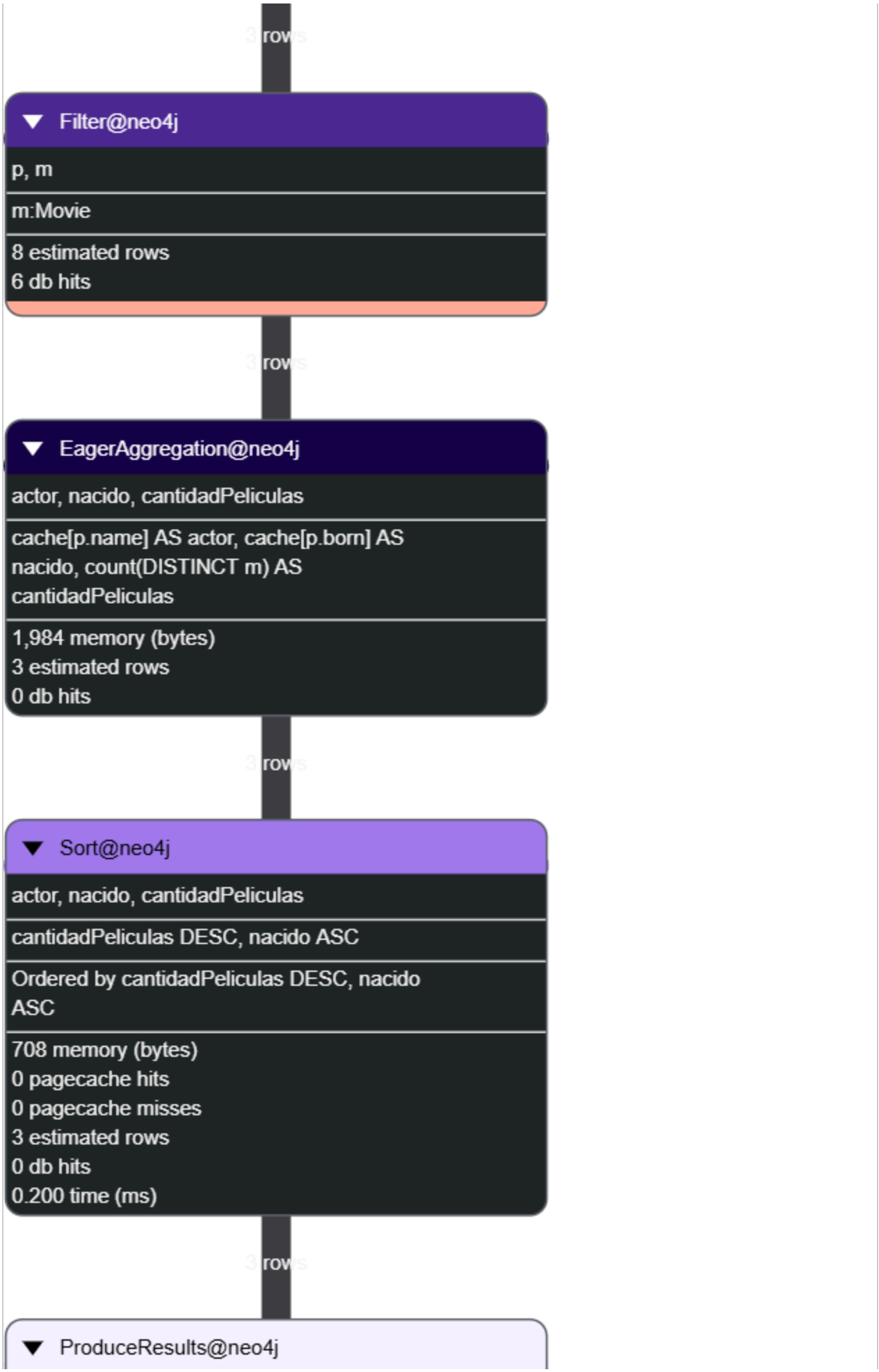

NodeByLabelScan@neo4j
p
p:Person
24 estimated rows
Filter@neo4j
p
cache[p.born] > $autoint_0
7 estimated rows
CacheProperties@neo4j
p
cache[p.name]
7 estimated rows
Expand(All)@neo4j
p, m
(p)-[:ACTED_IN]->(m)
8 estimated rows
Filter@neo4j
p, m
m:Movie
8 estimated rows
EagerAggregation@neo4j
actor, nacido, cantidadPeliculas
cache[p.name] AS actor, cache[p.born] AS

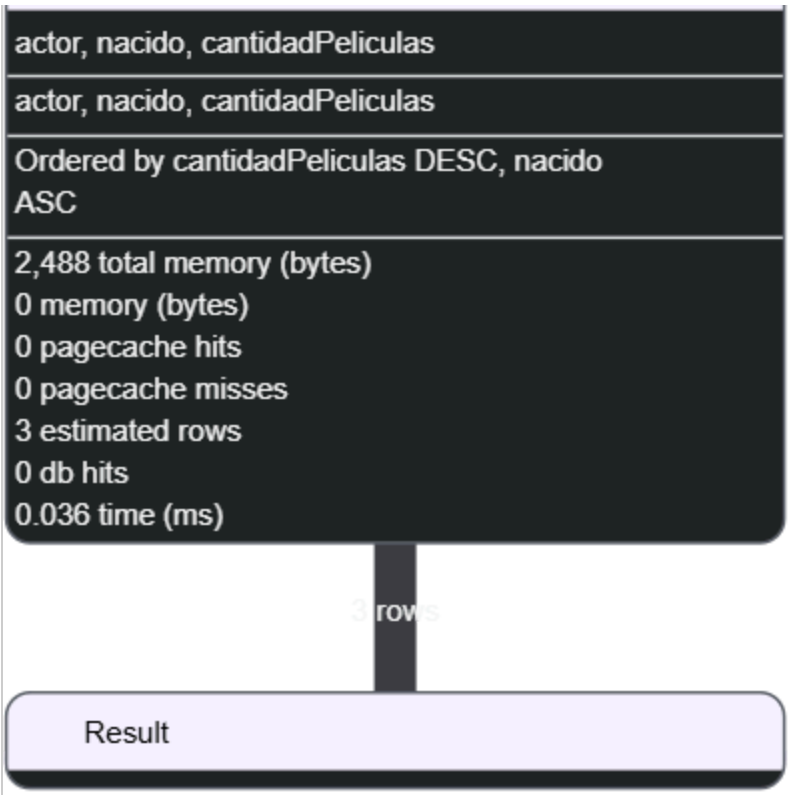



Cypher version: 25, Planner: COST, Runtime: PIPELINED

Total database accesses: 0, Total allocated memory: -, Total time: 51 ms

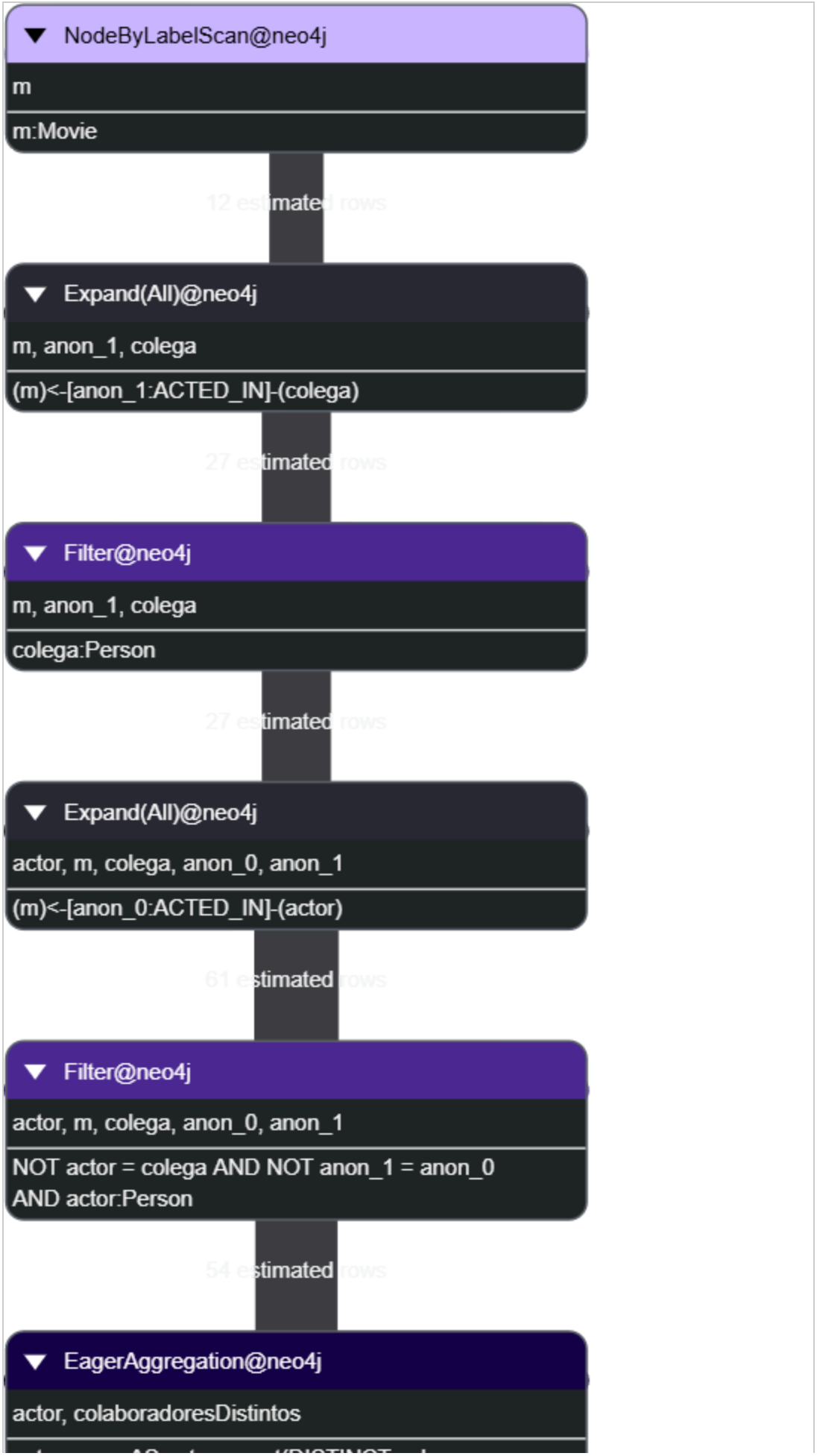

NodeByLabelScan@neo4j
p
p:Person
376 memory (bytes)
3 pagecache hits
0 pagecache misses
24 estimated rows
25 db hits
0.900 time (ms)
24 rows
Filter@neo4j
p
cache[p.born] > $autoint_0
7 estimated rows
24 db hits
row
CacheProperties@neo4j
p
cache[p.name]
7 estimated rows
6 db hits
row
Expand(All)@neo4j
p, m
(p)-[:ACTED_IN]->(m)
8 estimated rows
3 db hits

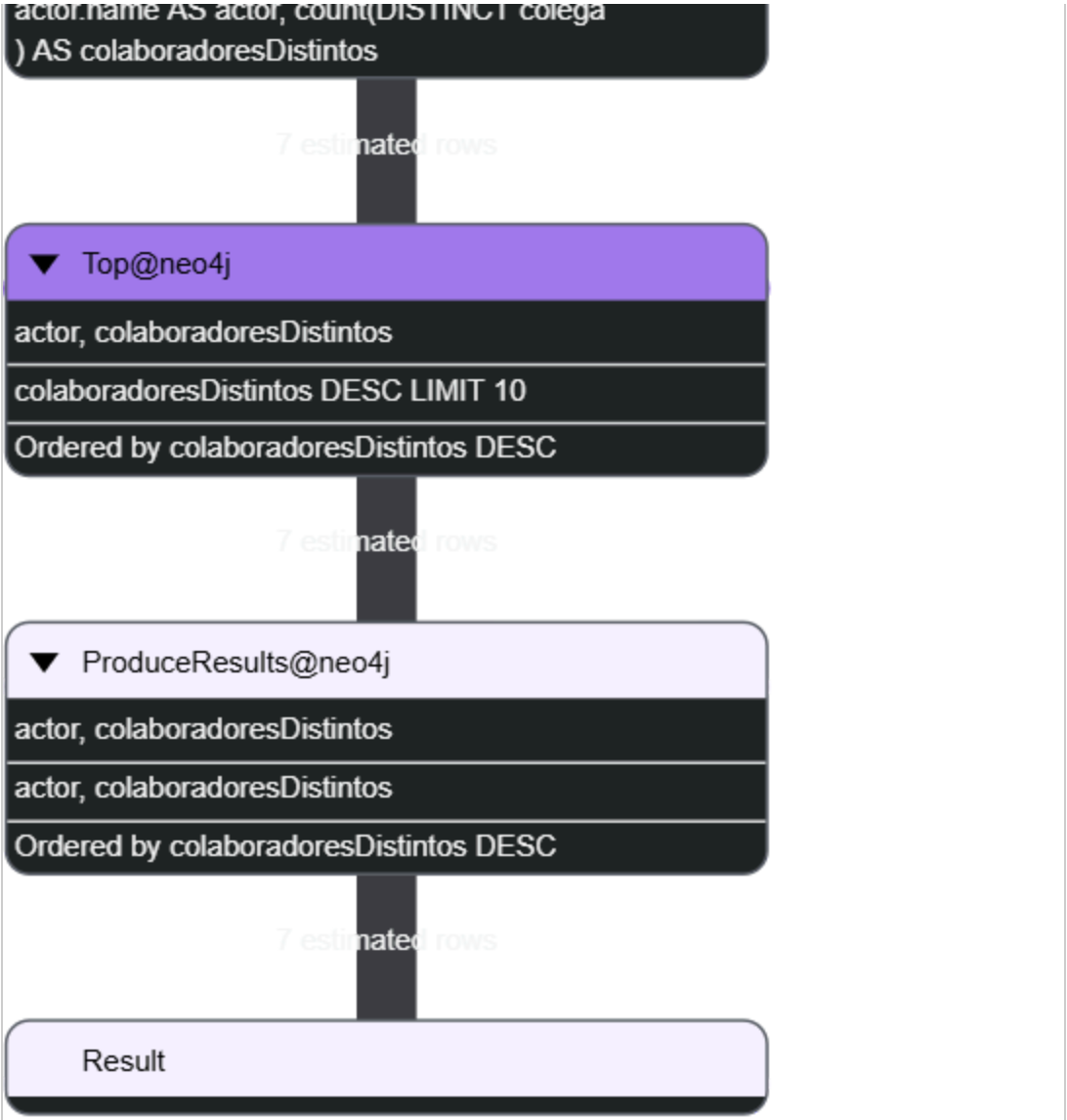

3 rows
Filter@neo4j
p, m
m:Movie
8 estimated rows
6 db hits
3 rows
EagerAggregation@neo4j
actor, nacido, cantidadPeliculas
cache[p.name] AS actor, cache[p.born] AS nacido, count(DISTINCT m) AS cantidadPeliculas
1,984 memory (bytes)
3 estimated rows
0 db hits
3 rows
Sort@neo4j
actor, nacido, cantidadPeliculas
cantidadPeliculas DESC, nacido ASC
Ordered by cantidadPeliculas DESC, nacido ASC
708 memory (bytes)
0 pagecache hits
0 pagecache misses
3 estimated rows
0 db hits
0.200 time (ms)
3 rows
ProduceResults@neo4j




Cypher version: 25, Planner: COST, Runtime: PIPELINED

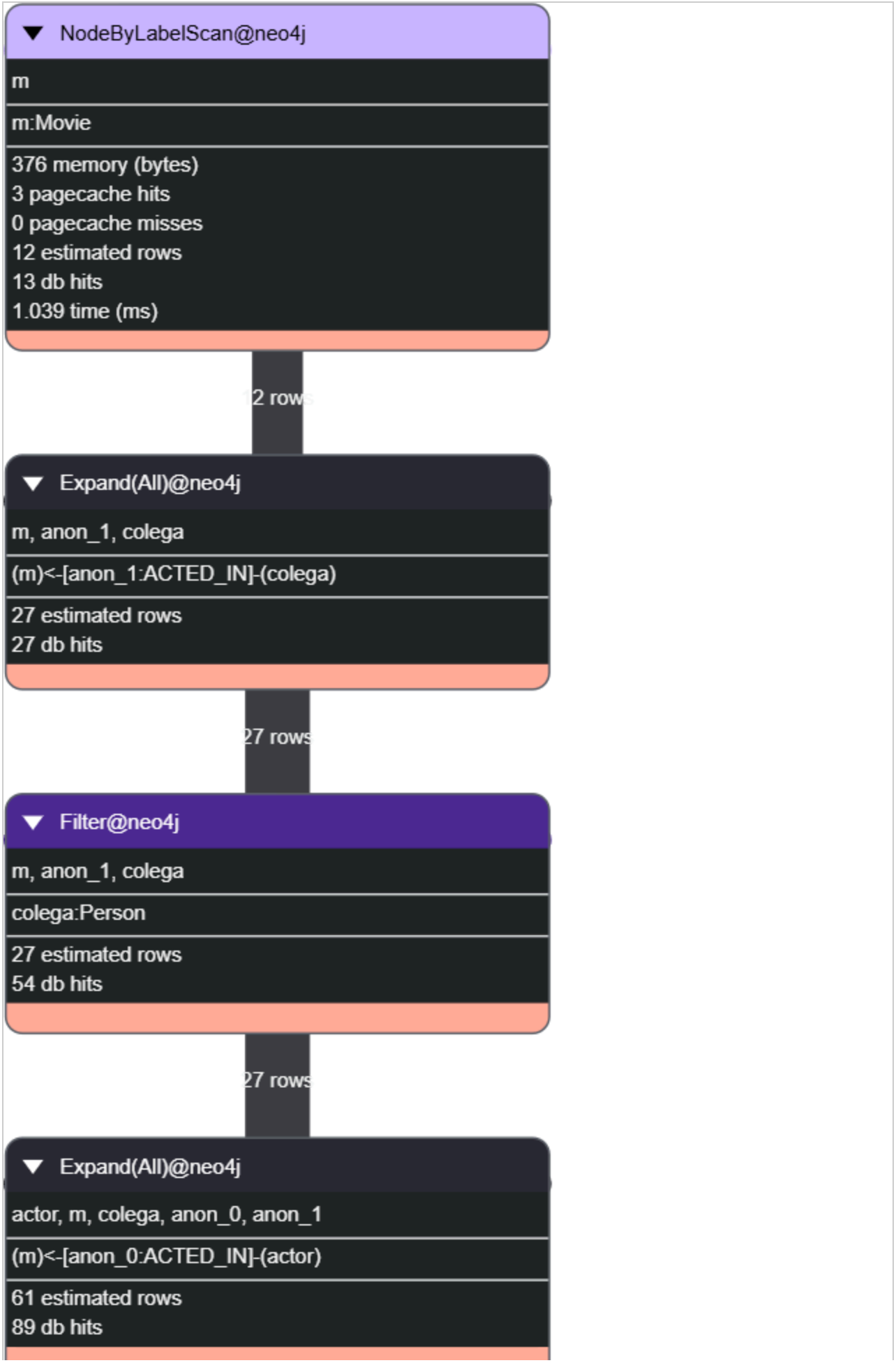
Total database accesses: 64, Total allocated memory: 2,488 bytes, Total time: 25 ms

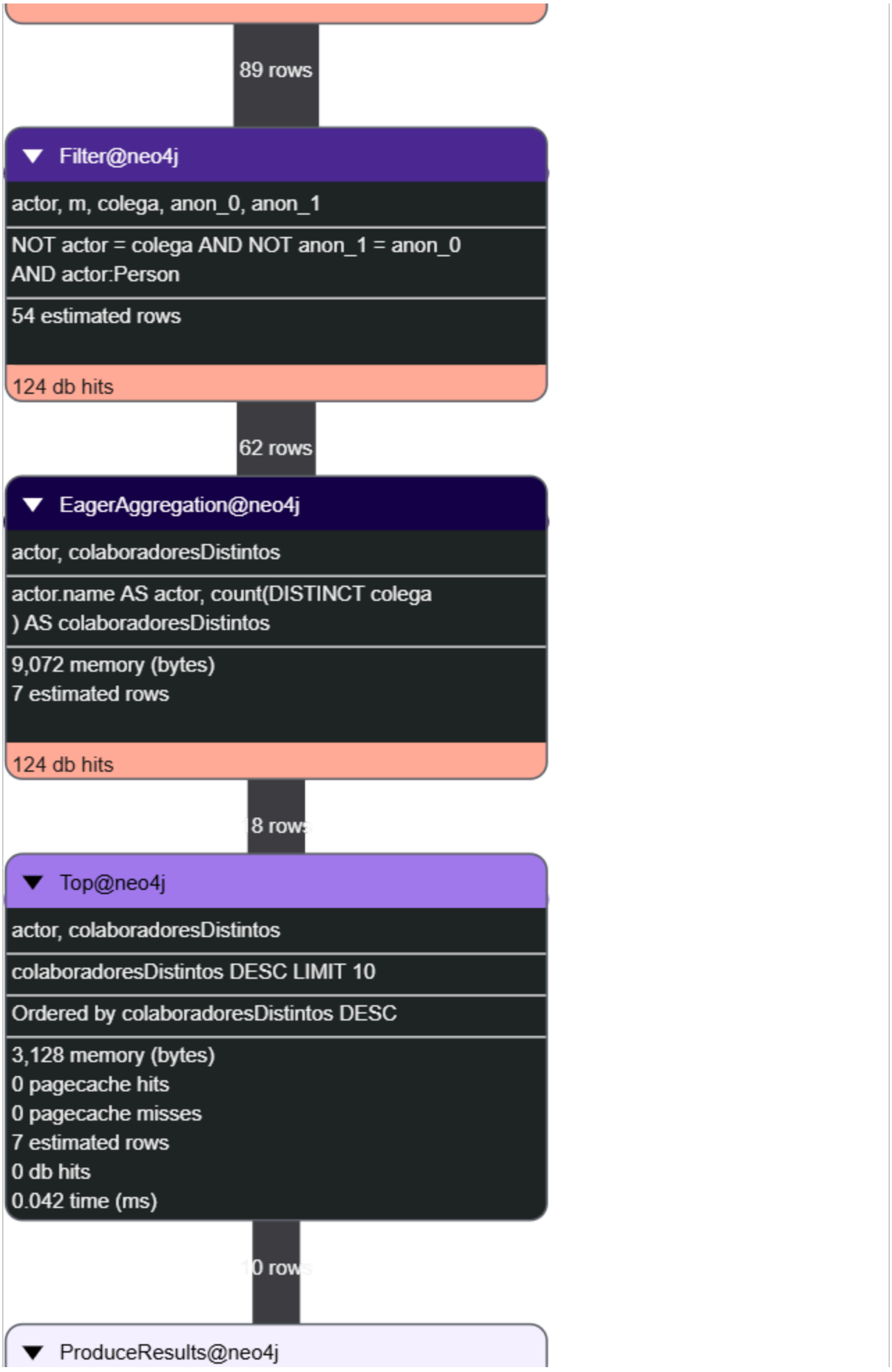

NodeByLabelScan@neo4j
m
m:Movie
12 estimated rows
Expand(All)@neo4j
m, anon_1, colega
(m)<-[anon_1:ACTED_IN]-(colega)
27 estimated rows
Filter@neo4j
m, anon_1, colega
colega:Person
27 estimated rows
Expand(All)@neo4j
actor, m, colega, anon_0, anon_1
(m)<-[anon_0:ACTED_IN]-(actor)
61 estimated rows
Filter@neo4j
actor, m, colega, anon_0, anon_1
NOT actor = colega AND NOT anon_1 = anon_0
AND actor:Person
54 estimated rows
EagerAggregation@neo4j
actor, colaboradoresDistintos

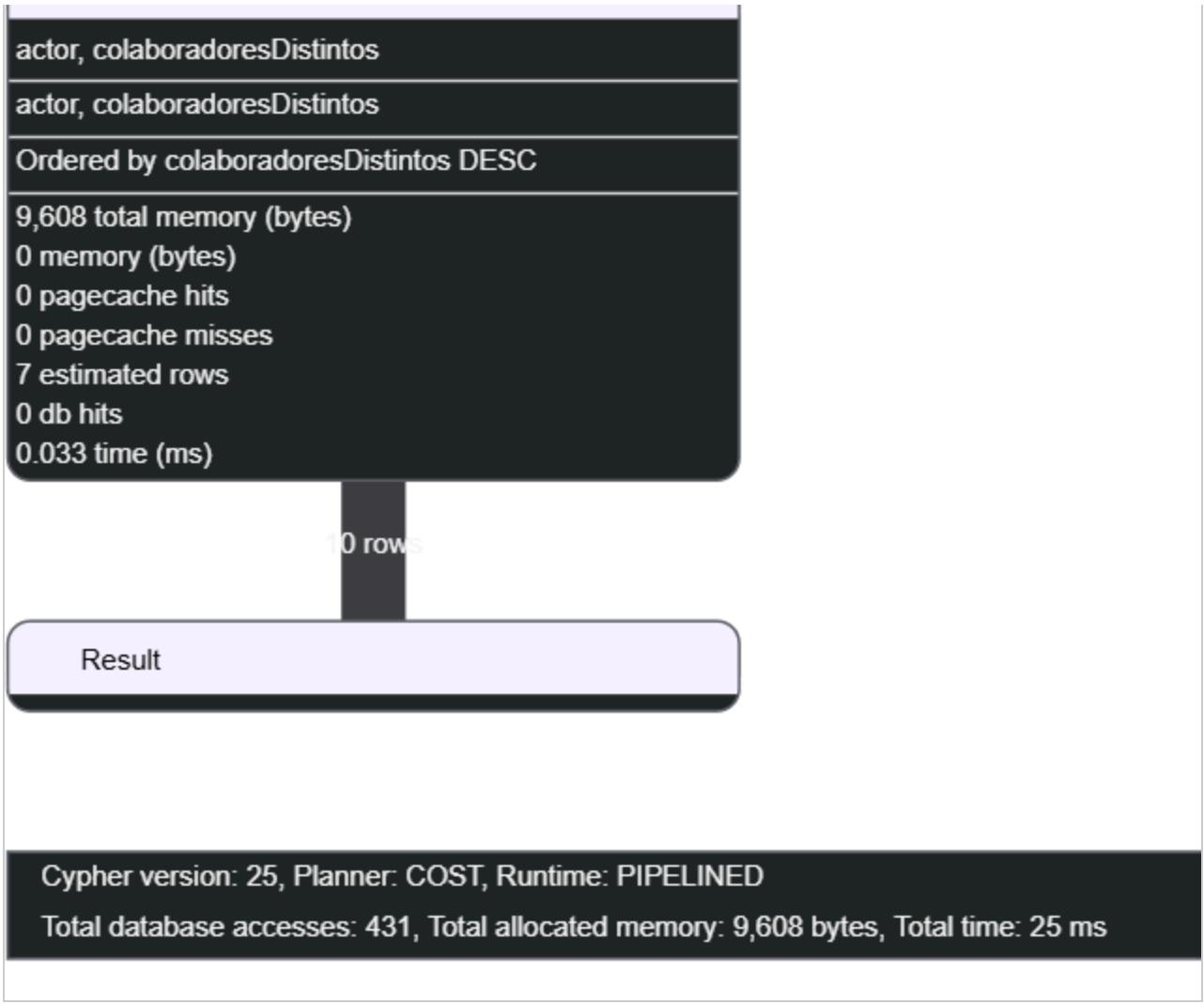



Cypher version: 25, Planner: COST, Runtime: PIPELINED

Total database accesses: 0, Total allocated memory: -, Total time: 11 ms


NodeByLabelScan@neo4j
m
m:Movie
376 memory (bytes)
3 pagecache hits
0 pagecache misses
12 estimated rows
13 db hits
1.039 time (ms)
2 rows
Expand(All)@neo4j
m, anon_1, colega
(m)<-[anon_1:ACTED_IN]-(colega)
27 estimated rows
27 db hits
27 rows
Filter@neo4j
m, anon_1, colega
colega:Person
27 estimated rows
54 db hits
27 rows
Expand(All)@neo4j
actor, m, colega, anon_0, anon_1
(m)<-[anon_0:ACTED_IN]-(actor)
61 estimated rows
89 db hits


89 rows
Filter@neo4j
actor, m, colega, anon_0, anon_1
NOT actor = colega AND NOT anon_1 = anon_0
AND actor:Person
54 estimated rows
124 db hits
62 rows
EagerAggregation@neo4j
actor, colaboradoresDistintos
actor.name AS actor, count(DISTINCT colega
) AS colaboradoresDistintos
9,072 memory (bytes)
7 estimated rows
124 db hits
8 rows
Top@neo4j
actor, colaboradoresDistintos
colaboradoresDistintos DESC LIMIT 10
Ordered by colaboradoresDistintos DESC
3,128 memory (bytes)
0 pagecache hits
0 pagecache misses
7 estimated rows
0 db hits
0.042 time (ms)
0 rows
ProduceResults@neo4j




## 5. Conclusiones

Cypher expresa recorridos y agregaciones de grafo de forma directa. OPTIONAL MATCH + collect, shortestPath, y MERGE para materializar conocimiento son los patrones clave. El orden Q1–Q15 antes de Q16 preserva reproducibilidad.